TPSI — GUIDA PRATICA COMPLETA V2

Come costruire un esercizio CRUD + Grafici

JSON-Server · jQuery AJAX · Bootstrap 5 · Chart.js · Tutto in LOCALE

⚠️ **VERIFICA OFFLINE:** usa SEMPRE le librerie locali — niente CDN online!

📁 STRUTTURA DEI FILE

```
MioProgetto/
├── server/
│   ├── db.json              ← il "database" finto
│   ├── Start.bat            ← avvia json-server (doppio click)
│   └── New_installation.bat ← installa json-server (solo prima volta)
│
├── index.html       ← pagina web + struttura Bootstrap
├── index.js         ← tutta la logica JS (CRUD + grafici)
├── libreria.js      ← funzioni riutilizzabili (copiala sempre)
│
├── jquery.js        ← jQuery LOCALE ← questa è già in ../AJAX/
└── chart.js         ← Chart.js LOCALE ← questa è già in ../AJAX/
```

⚠️ **In verifica non hai rete.** Tutte le librerie vanno nella cartella del progetto (o referenziate con path relativo se già presenti).

FASE 1 — db.json

```
{
  "studenti": [
    { "id": 1, "nome": "Filippo", "cognome": "Rossi", "classe": "3B", "voto": 8 },
    { "id": 2, "nome": "Giulia",  "cognome": "Bianchi","classe": "4A", "voto": 9 },
    { "id": 3, "nome": "Marco",   "cognome": "Verdi",  "classe": "3B", "voto": 6 }
  ]
}
```

Regole:

- L' `id` deve essere **numero** (non stringa `"1"`, ma `1`)
- Il nome della chiave (`"studenti"`) diventa l'endpoint: `http://localhost:3000/studenti`
- json-server genera l'id automaticamente nei POST → non includerlo nel body della richiesta

---

FASE 2 — Start.bat

```
cd /d %~dp0
json-server --watch db.json --port 3000
pause
```

Flusso di avvio:

1. Doppio click su `Start.bat`
2. Lascia il terminale aperto
3. Verifica: apri il browser su `http://localhost:3000/studenti` → vedi il JSON? ✅ Funziona.

FASE 3 — libreria.js (identica per ogni esercizio)

```
"use strict";

function inviaRichiesta(method, url, parameters = {}) {
    let contentType;
    if (method.toUpperCase() == "GET")
        contentType = "application/x-www-form-urlencoded;charset=utf-8";
    else {
        contentType = "application/json; charset=utf-8";
        parameters = JSON.stringify(parameters); // converte oggetto JS → stringa JSON
    }
    return $.ajax({        // ritorna una "promessa" jqXHR
        "url": url,
        "data": parameters,
        "type": method,
        "contentType": contentType,
        "dataType": "json",
        "timeout": 5000,
```

```
    });
}

function errore(jqXHR, textStatus, stringError) {
    if (jqXHR.status == 0)
        alert("Server non raggiunto. Hai avviato Start.bat?");
    else if (jqXHR.status == 404)
        alert("404: risorsa non trovata. Controlla l'URL e il nome in db.json.");
    else
        alert("Errore " + jqXHR.status + ": " + jqXHR.responseText);
}
```

FASE 4 — index.html

```html
<!DOCTYPE html>
<html lang="it">
<head>
    <meta charset="UTF-8">
    <title>Gestione Studenti</title>
    <!-- Bootstrap CSS — locale in verifica! -->
    <link href="bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container mt-4">

    <h1 class="text-center mb-4">📚 Gestione Studenti</h1>

    <!-- Alert feedback (nascosto con d-none) -->
    <div id="divAlert" class="alert d-none" role="alert"></div>

    <!-- Form inserimento / modifica -->
    <div class="card mb-4">
        <div class="card-header">
            <h5 id="titoloForm">➕ Inserisci nuovo studente</h5>
        </div>
        <div class="card-body">
            <input type="hidden" id="txtId">
            <div class="row g-3">
                <div class="col-md-3">
                    <label class="form-label">Nome</label>
                    <input type="text" id="txtNome" class="form-control">
                </div>
                <div class="col-md-3">
                    <label class="form-label">Cognome</label>
                    <input type="text" id="txtCognome" class="form-control">
                </div>
                <div class="col-md-3">
                    <label class="form-label">Classe</label>
                    <input type="text" id="txtClasse" class="form-control">
                </div>
                <div class="col-md-3">
                    <label class="form-label">Voto</label>
                    <input type="number" id="txtVoto" class="form-control" min="1" max="10">
                </div>
            </div>
            <div class="mt-3 d-flex gap-2">
                <button class="btn btn-success" id="btnInserisci"
                        onclick="inserisciStudente()">💾 Inserisci</button>
                <button class="btn btn-warning d-none" id="btnAggiorna"
                        onclick="aggiornaStudente()">✏️ Aggiorna</button>
                <button class="btn btn-secondary d-none" id="btnAnnulla"
                        onclick="annullaModifica()">✕ Annulla</button>
            </div>
```

```html
        </div>
    </div>

    <!-- Layout a due colonne: Tabella + Grafici -->
    <div class="row">
        <div class="col-lg-7">
            <!-- Tabella -->
            <div class="card">
                <div class="card-header d-flex justify-content-between align-items-center">
                    <h5 class="mb-0">📋 Lista Studenti</h5>
                    <button class="btn btn-primary btn-sm"
                            onclick="leggiStudenti()">🔄 Ricarica</button>
                </div>
                <div class="card-body p-0">
                    <table class="table table-hover table-striped mb-0">
                        <thead class="table-dark">
                            <tr>
                                <th>ID</th><th>Nome</th><th>Cognome</th>
                                <th>Classe</th><th>Voto</th><th>Azioni</th>
                            </tr>
                        </thead>
                        <tbody id="tbodyStudenti"></tbody>
                    </table>
                </div>
            </div>
        </div>
        <div class="col-lg-5">
            <!-- Grafico 1: barre per classe -->
            <div class="card mb-3">
                <div class="card-header">📊 Studenti per Classe</div>
                <div class="card-body">
                    <canvas id="canvasClassi"></canvas>
                </div>
            </div>
            <!-- Grafico 2: ciambella distribuzione voti -->
            <div class="card">
                <div class="card-header">🍩 Distribuzione Voti</div>
                <div class="card-body">
                    <canvas id="canvasVoti"></canvas>
                </div>
            </div>
        </div>
    </div>

</div>

<!-- LIBRERIE LOCALI — ordine obbligatorio: jQuery prima di tutto! -->
<script src="jquery.js"></script>
<script src="bootstrap.bundle.min.js"></script>
<script src="chart.js"></script>
<script src="libreria.js"></script>
<script src="index.js"></script>
```

```
</body>
</html>
```

FASE 5 — index.js — BLOCCO 1: Setup e Grafici

```
"use strict";
const URL_BASE = "http://localhost:3000";

// Variabili globali per i grafici.
// Devono essere globali per poterli aggiornare da tutte le funzioni.
let graficoClassi;
let graficoVoti;

$(document).ready(function () {
    creaGrafici();    // crea i canvas vuoti
    leggiStudenti();  // carica i dati dal server
});

function creaGrafici() {
    // --- Grafico 1: Barre ---
    graficoClassi = new Chart($("#canvasClassi"), {
        type: "bar",
        data: {
            labels: [],     // ← vuoto ora, popolato da aggiornaGrafici()
            datasets: [{
                label: "Numero studenti",
                data: [],   // ← vuoto ora
                backgroundColor: [
                    "rgba(54,162,235,0.7)", "rgba(255,99,132,0.7)",
                    "rgba(75,192,192,0.7)", "rgba(255,206,86,0.7)"
                ],
                borderWidth: 2
            }]
        },
        options: {
            responsive: true,
            scales: { y: { beginAtZero: true, ticks: { stepSize: 1 } } }
        }
    });

    // --- Grafico 2: Ciambella ---
    graficoVoti = new Chart($("#canvasVoti"), {
        type: "doughnut",
        data: {
            labels: [],
            datasets: [{
                data: [],
                backgroundColor: [
                    "rgba(255,99,132,0.8)","rgba(255,159,64,0.8)",
                    "rgba(255,205,86,0.8)","rgba(75,192,192,0.8)",
                    "rgba(54,162,235,0.8)"
                ],
                borderWidth: 2
            }]
```

```
        },
        options: { responsive: true, plugins: { legend: { position: "bottom" } } }
    });
}

function aggiornaGrafici(studenti) {
    // --- Conta studenti per classe ---
    let classi = {};
    for (let i = 0; i < studenti.length; i++) {
        let c = studenti[i].classe;
        classi[c] = classi[c] ? classi[c] + 1 : 1;
    }
    graficoClassi.data.labels = Object.keys(classi);
    graficoClassi.data.datasets[0].data = Object.values(classi);
    graficoClassi.update(); // ← OBBLIGATORIO per ridisegnare!

    // --- Conta studenti per voto ---
    let voti = {};
    for (let i = 0; i < studenti.length; i++) {
        let v = "Voto " + studenti[i].voto;
        voti[v] = voti[v] ? voti[v] + 1 : 1;
    }
    let votiOrd = Object.keys(voti).sort();
    graficoVoti.data.labels = votiOrd;
    graficoVoti.data.datasets[0].data = votiOrd.map(k => voti[k]);
    graficoVoti.update(); // ← OBBLIGATORIO per ridisegnare!
}
```

FASE 5 — BLOCCO 2: GET

```
function leggiStudenti() {
    let request = inviaRichiesta("GET", URL_BASE + "/studenti");
    request.fail(errore);
    request.done(function (studenti) {
        // Svuota e ripopola la tabella
        $("#tbodyStudenti").html("");
        for (let i = 0; i < studenti.length; i++) {
            let s = studenti[i];
            let tr = $("<tr>").appendTo("#tbodyStudenti");
            $("<td>").text(s.id).appendTo(tr);
            $("<td>").text(s.nome).appendTo(tr);
            $("<td>").text(s.cognome).appendTo(tr);
            $("<td>").text(s.classe).appendTo(tr);
            $("<td>").text(s.voto).appendTo(tr);

            let tdAz = $("<td>").appendTo(tr);
            $("<button>").addClass("btn btn-warning btn-sm me-1")
                .text("✏️ Modifica")
                .data("id", s.id)         // salva l'id sul pulsante
                .on("click", mostraFormModifica)
                .appendTo(tdAz);
```

```
            $("<button>").addClass("btn btn-danger btn-sm")
                .text("🗑 Elimina")
                .data("id", s.id)
                .on("click", eliminaStudente)
                .appendTo(tdAz);
        }
        // ← Aggiorna ANCHE i grafici con gli stessi dati!
        aggiornaGrafici(studenti);
    });
}
```

FASE 5 — BLOCCO 3: POST

```
function inserisciStudente() {
    // 1. Leggi i dati dal form
    let nome    = $("#txtNome").val().trim();
    let cognome = $("#txtCognome").val().trim();
    let classe  = $("#txtClasse").val().trim();
    let voto    = parseInt($("#txtVoto").val());

    // 2. Valida PRIMA di inviare — se sbagliato, esci subito con return
    if (nome === "" || cognome === "" || classe === "") {
        mostraAlert("danger", "⚠️ Compila tutti i campi!"); return;
    }
    if (isNaN(voto) || voto < 1 || voto > 10) {
        mostraAlert("danger", "⚠️ Voto non valido!"); return;
    }

    // 3. Crea l'oggetto — SENZA id (json-server lo genera)
    let nuovo = { nome: nome, cognome: cognome, classe: classe, voto: voto };

    // 4. Invia il POST
    let request = inviaRichiesta("POST", URL_BASE + "/studenti", nuovo);
    request.fail(errore);
    request.done(function (creato) {
        mostraAlert("success", "✅ Inserito con id: " + creato.id);
        svuotaForm();
        leggiStudenti(); // ricarica tabella + grafici
    });
}
```

FASE 5 — BLOCCO 4: mostraFormModifica + PATCH

```
// Pre-compila il form per la modifica (GET singolo)
function mostraFormModifica() {
    let id = $(this).data("id");
    let request = inviaRichiesta("GET", URL_BASE + "/studenti/" + id);
    request.fail(errore);
    request.done(function (s) {
        $("#txtId").val(s.id);
        $("#txtNome").val(s.nome);
        $("#txtCognome").val(s.cognome);
        $("#txtClasse").val(s.classe);
        $("#txtVoto").val(s.voto);
        $("#titoloForm").text("✏️ Modifica studente (id: " + s.id + ")");
        $("#btnInserisci").addClass("d-none");
        $("#btnAggiorna, #btnAnnulla").removeClass("d-none");
        $("html, body").animate({ scrollTop: 0 }, 300);
    });
}
```

```
// PATCH — aggiorna solo i campi inviati
function aggiornaStudente() {
    let id      = $("#txtId").val();
    let nome    = $("#txtNome").val().trim();
    let cognome = $("#txtCognome").val().trim();
    let classe  = $("#txtClasse").val().trim();
    let voto    = parseInt($("#txtVoto").val());

    if (nome === "" || cognome === "" || classe === "") {
        mostraAlert("danger", "⚠️ Compila tutti i campi!"); return;
    }

    // L'id va nell'URL (/studenti/3), NON nel body
    let request = inviaRichiesta("PATCH", URL_BASE + "/studenti/" + id,
                                 { nome, cognome, classe, voto });
    request.fail(errore);
    request.done(function () {
        mostraAlert("success", "✅ Aggiornato!");
        annullaModifica();
        leggiStudenti();
    });
}
```

FASE 5 — BLOCCO 5: DELETE + Supporto

```
// DELETE — elimina (id solo nell'URL, niente body)
function eliminaStudente() {
    let id = $(this).data("id");
    if (!confirm("Eliminare lo studente con id " + id + "?")) return;

    let request = inviaRichiesta("DELETE", URL_BASE + "/studenti/" + id);
    request.fail(errore);
    request.done(function () {
        mostraAlert("success", "🗑️ Eliminato!");
        leggiStudenti();
    });
}

// Torna alla modalità inserimento
function annullaModifica() {
    svuotaForm();
    $("#titoloForm").text("➕ Inserisci nuovo studente");
    $("#btnInserisci").removeClass("d-none");
    $("#btnAggiorna, #btnAnnulla").addClass("d-none");
}

function svuotaForm() {
    $("#txtId, #txtNome, #txtCognome, #txtClasse, #txtVoto").val("");
}
```

```
// Mostra un Alert Bootstrap colorato
function mostraAlert(tipo, messaggio) {
    let div = $("#divAlert");
    div.removeClass("alert-success alert-danger alert-warning");
    div.addClass("alert-" + tipo).text(messaggio).removeClass("d-none");
    setTimeout(function () { div.addClass("d-none"); }, 4000);
}
```

SCHEMA RAPIDO — Tutte le operazioni

Metodo	URL	Body	Risultato
GET	`/studenti`	nessuno	Array di tutti gli studenti
GET	`/studenti/3`	nessuno	Singolo oggetto studente
POST	`/studenti`	{ nome, cognome, classe, voto }	Studente creato (con id generato)
PATCH	`/studenti/3`	Solo i campi da cambiare	Studente aggiornato parzialmente
DELETE	`/studenti/3`	nessuno	Record eliminato

CHART.JS — Riferimento rapido

type	Grafico
`"bar"`	Barre verticali
`"line"`	Linee
`"doughnut"`	Ciambella
`"pie"`	Torta
`"polarArea"`	Raggi circolari
`"radar"`	Radar poligonale

Pattern da memorizzare:

```
// 1. Dichiara la variabile GLOBALE
let mioGrafico;

// 2. Crealo UNA sola volta in $(document).ready o in una funzione init
mioGrafico = new Chart($("#idCanvas"), { type: "bar", data: { labels:[], datasets:[{data:[]}] }
});

// 3. Ogni volta che cambiano i dati: aggiorna e chiama .update()
mioGrafico.data.labels = ["A","B","C"];
mioGrafico.data.datasets[0].data = [10, 20, 5];
mioGrafico.update(); // ← senza questo, il grafico non si ridisegna!
```

CHECKLIST VERIFICA ✅

- ☐ `jquery.js` è nella cartella del progetto (o path relativo corretto)?
- ☐ `chart.js` è nella cartella del progetto (o path relativo corretto)?
- ☐ Bootstrap locale (`bootstrap.min.css` + `bootstrap.bundle.min.js`)?
- ☐ `Start.bat` è stato avviato? Il terminale è ancora aperto?
- ☐ `http://localhost:3000/nomeCollection` risponde con JSON nel browser?
- ☐ Ogni `inviaRichiesta()` ha sia `.fail(errore)` che `.done()` ?
- ☐ La validazione avviene **prima** di `inviaRichiesta()` ?
- ☐ Nel POST il body **non contiene** l'id?
- ☐ Nel PATCH/PUT/DELETE l'id è **nell'URL**, non nel body?
- ☐ Dopo ogni modifica viene richiamata la funzione che ricarica la tabella?
- ☐ I grafici vengono aggiornati con i nuovi dati e poi `.update()` viene chiamato?
- ☐ Console F12 senza errori rossi?